

Full System Architecture Blueprint

Partner Truck Visibility & Logistics

Coordination Platform

This document provides the complete technical and operational architecture of a logistics coordination platform designed for manufacturers, logistics managers, dispatchers, trucking partners, and drivers.

The platform centralizes:

- Truck visibility
- Dispatch operations
- Partner coordination
- GPS tracking
- Operational analytics
- Realtime communication
- Multi-tenant SaaS infrastructure

The architecture is designed to scale from a single manufacturing company into a regional logistics network.

1. High-Level System Architecture

The platform consists of the following major layers:

1. Frontend Layer

- Web dashboards
- Mobile driver app
- Partner portal
- Admin panel

2. API Layer

- Authentication APIs
- Dispatch APIs
- Tracking APIs
- Analytics APIs

3. Core Business Logic Layer

- Dispatch engine
- Matching engine
- Status engine
- Notification engine

4. Realtime Infrastructure

- WebSockets
- GPS streaming
- Event queues

5. Database Layer

- Operational database

- Analytics database
- Logs

6. External Services

- Maps
- SMS
- Email
- Push notifications

2. User Roles & Permissions

Role	Main Responsibilities
Super Admin	Manage platform, companies, subscriptions
Logistics Manager	Monitor trucks, assign dispatches, analytics
Dispatcher	Assign deliveries and monitor trips
Partner Admin	Manage trucking fleets and drivers
Driver	Receive jobs and update statuses

3. Database Architecture

Core relational database entities:

- companies
- partners
- trucks
- drivers
- dispatches
- routes
- deliveries
- tracking_logs
- maintenance_records
- notifications
- users
- permissions
- audit_logs

Key database relationships:

- One company can have multiple dispatches
- One partner can own multiple trucks
- One truck can have multiple trips
- One driver can handle multiple dispatches

4. Core Database Tables

Table: trucks
id

partner_id
plate_number
truck_type
capacity
status
gps_latitude
gps_longitude
last_updated

Table: dispatches
id
truck_id
driver_id
origin
destination
cargo_type
status
assigned_at

5. Realtime Operational Flow

STEP 1:

Partner updates truck availability.

STEP 2:

Truck status is pushed into the realtime engine.

STEP 3:

Dashboard updates instantly using WebSockets.

STEP 4:

Logistics manager assigns delivery.

STEP 5:

Dispatch engine validates availability.

STEP 6:

Driver receives trip assignment.

STEP 7:

GPS tracking begins.

STEP 8:

Delivery lifecycle events are streamed:

- Loading
- In Transit
- Arrived

- Delivered

STEP 9:

Analytics engine records operational metrics.

6. Realtime Infrastructure

Realtime architecture is critical because logistics operations require live visibility.

Recommended technologies:

- Socket.IO
- Redis Pub/Sub
- Supabase Realtime
- WebSockets

Realtime events:

- truck.status.updated
- dispatch.created
- trip.started
- gps.updated
- delivery.completed
- maintenance.alert

7. API Architecture

Suggested REST API Structure:

Authentication:

POST /api/auth/login

POST /api/auth/register

Truck APIs:

GET /api/trucks

POST /api/trucks

PATCH /api/trucks/{id}/status

Dispatch APIs:

POST /api/dispatches

GET /api/dispatches

PATCH /api/dispatches/{id}

Tracking APIs:

POST /api/gps/update

GET /api/trips/live

Analytics APIs:

GET /api/analytics/utilization

GET /api/analytics/delays

8. Security Architecture

Security mechanisms:

- JWT authentication
- Role-based access control
- Multi-tenant isolation
- HTTPS encryption
- Audit logs
- API rate limiting
- Secure GPS event validation
- Backup and disaster recovery

Critical security consideration:

One company's operational data must never be visible to another company.

9. Scalability Strategy

Phase 1:

Single company deployment.

Phase 2:

Multiple trucking partners.

Phase 3:

Regional logistics ecosystem.

Phase 4:

AI-powered orchestration platform.

Horizontal scaling considerations:

- Load-balanced API servers
- Redis caching
- Queue workers
- Event-driven architecture
- CDN for assets

10. Deep Operational Pain Points

- No centralized truck visibility
- Manual dispatch coordination
- Viber and Messenger dependency
- Dispatch delays
- Idle truck inefficiency
- Poor fleet utilization
- Double booking of trucks
- No realtime operational intelligence
- No analytics dashboards
- No predictive forecasting

- Poor maintenance visibility
- Lack of partner coordination
- Warehouse congestion
- Fuel inefficiency
- Human dependency on experienced dispatchers

11. Future Expansion Features

Potential advanced modules:

- AI dispatch recommendations
- Predictive maintenance
- Return-load matching
- Dynamic pricing
- Smart routing
- Driver behavior analytics
- Fuel optimization
- IoT truck sensors
- Automated invoicing
- ERP integrations
- Port and ferry integrations

12. Strategic Startup Opportunity

The strongest strategic advantage of this platform is industry specificity.

The platform does not compete directly with:

- Lalamove
- Ninja Van
- LBC

Instead, it focuses on:

- Enterprise fleet visibility
- Partner coordination
- Internal logistics orchestration
- Operational intelligence

This creates:

- Strong B2B SaaS opportunities
- Recurring revenue
- High switching costs
- Network effects
- Regional scalability

13. Final System Vision

The long-term vision is to become the operational backbone of regional logistics coordination.

The platform evolves from:

Truck visibility dashboard

→ Dispatch coordination platform

→ Logistics intelligence platform

→ Regional logistics network

This architecture supports:

- Manufacturing logistics
- Inter-island cargo
- Construction logistics
- Feed and flour distribution
- Warehouse coordination
- Shared trucking ecosystems